YENEPOYA INSTITUTE OF ARTS,SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENTUNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

HIGH LEVEL DESIGN

ON

OPTIMIZATION STRATEGIES FOR NATURAL LANGUAGE INTERFACE FOR EDTECH INFRASTRUCTURE

SUBMITTED BY

SWALIH BIN HAMEED

REGNO :

23BCAICD105

LH-06

GUIDED BY

MISS DHEEKSHA SHETTY

SUBMISSION DATE

25-04-2026

Table of Contents

1. Introduction
1.1 Objective
1.2 Scope
1.3 Definitions
2. System Overview

3. Architectural Design
3.1 Architecture
3.2 High-Level Components
3.3 Architecture Diagram Components

4. Module Design
4.1CLIInterface Module
4.2 Input Processing Module
4.3 Keyword Matching Module
4.4 Response Management Module

5. Data Flow and Control Flow
6. Deployment Architecture
7. Technology Stack

8. System Characteristics
8.1 Scalability
8.2 Reliability
8.3 Maintainability

9. Assumptions and Dependencies
10. Constraints and Limitations

11. Non-Functional Requirements
11.1 Performance
11.2 Security
11.3 Usability

12. Future Enhancements
13. Conclusion

1. Introduction

1.1 Objective

Theobjective of thissystem is to design and develop a command-line chatbot capable of handling basic EdTech-related queries using simple string matching techniques in Python. The system aims to provide a foundational understanding of chatbot architecture and natural language interfaces.

1.2 Scope

Thisdocument covers the high-level architecture, system components, and interaction mechanisms of the chatbot. It focuses on rule-based query handling and excludes advanced NLP or machine learning features.

1.3 Definitions

- **CLI (CommandLineInterface):** A text-based interface for interacting with software
- **Chatbot:** A system that simulates conversation with users
- **Keyword Matching:** Identifying relevant words in user input to determine intent

2. System Overview

Thesystem isa lightweightCLI-based chatbot that allows users to interact using simple text commands such as:

- “help”
- “show status”
- “courses”

The chatbot processes these inputs using basic string matching and returns predefined responses relevant to an EdTech environment.

Key Features:

- Simple interaction model
- Fast response time

- Offline functionality
- Easy extensibility

3. Architectural Design

3.1 Architecture Style

The system follows a **Layered Architecture**, ensuring separation of concerns and modularity.

3.2 High-Level Components

- Presentation Layer (CLI Interface)
- Application Layer (Processing Logic)
- Data Layer (Response Storage)

3.3 Architecture Diagram

```
 +----------------------+
|   Presentation Layer |
|     (CLI Input)       |
+----------+-----------+
           |
           v
 +----------------------+
|    Application Layer |
| (Parsing & Matching) |
+----------+-----------+
           |
           v
+----------------------+
| Data Layer|
| (Static Responses)|
+-- --- -- --- --- -- --- -- -- +
```

4. Module Design

4.1 CLI Interface Module

Handles all user interactions.

Responsibilities:

- Accept user input
- Display chatbot responses
- Maintain continuous interaction loop

4.2 Input Processing Module

Processes raw input for consistency.

Functions:

- Convert text to lowercase
- Remove extra spaces
- Normalize input

4.3 Keyword Matching Module

Core logic of the system.

Approach:

- Uses if-else conditions or dictionary mapping
- Matches keywords such as “help”, “status”, “courses”

4.4 Response Management Module

Stores and retrieves predefined responses.

Example Structure:

```
responses = {
    "help": "Available commands...",
    "status": "System is running",
    "courses": "Available courses..."
}
```

5. Data Flow and Control Flow

Data Flow:

1. User inputs command
2. Input is processed
3. Keywords are extracted
4. Matching logic identifies intent
5. Response is retrieved
6. Output is displayed

Control Flow:

- Continuous loop until exit command is issued

6. Deployment Architecture

- Deployed locally on user machine
- Runs via Python interpreter
- Executed through terminal or VS Code

7. Technology Stack

- **Programming Language:** Python
- **Interface:** CLI
- **IDE:** VS Code
- **Libraries:** StandardPythonlibraries

8. System Characteristics

8.1 Scalability

Limited scalability due to rule-based logic.

8.2 Reliability

Highlyreliableduetominimaldependencies and simple design.

8.3 Maintainability

Easy to maintain and extend by adding new keywords and responses.

9. Assumptions and Dependencies

- Users provide simple, clear inputs
- No spelling correction required
- No external APIs or databases

10. Constraints and Limitations

- Cannot handle complex queries
- No contextual understanding
- No learning capability
- Limited flexibility

11. Non-Functional Requirements

11.1 Performance

- Response time < 100ms

- Lightweight execution

11.2 Security

- No sensitive data processing
- Basic input validation

11.3 Usability

- Easy-to-use CLI interface
- Minimal learning curve

12. Future Enhancements

- Integration of NLP techniques
- GUI or web-based interface
- Database integration
- Context-aware responses
- Voice-based interaction

13. Conclusion

This system provides a foundational implementation of a chatbot using basic string matching techniques. It demonstrates essential concepts such as input processing, modular design, and user interaction in a simple and efficient manner, making it ideal for learning and prototyping EdTech solutions.